Breaking the Loop: A Graph-Based Approach to Music Discovery

Allan Minn, Andrew Moore, Belinda Xu, Ridge Coughlin
UC Berkeley Master of Information and Data Science

Abstract
Recommendation systems are effective at personalization, but they often reinforce existing preferences rather than expanding them. This project presents a graph-based approach to music discovery that models songs as nodes connected by acoustic similarity and generates listening paths that move a user from familiar tracks into new sonic territory. By identifying songs that sit between clusters and routing recommendations through them, the system produces transitions that feel gradual rather than abrupt.

Introduction
Spotify's recommendation system is one of the most widely cited examples of personalization at scale. It combines collaborative filtering with content-based methods to surface music that aligns closely with a listener's taste [1].

Over time, this creates a feedback loop. As the system becomes more confident in what a user likes, it continues to recommend similar content. Familiarity drives engagement, so recommendations converge instead of expanding. Prior work has shown that algorithmic recommendations can reduce consumption diversity over time [2].
This project approaches recommendation differently. Instead of asking what is most similar to a given song, we ask how to move a listener from one part of the space to another in a way that feels natural.

Data and Preprocessing
We work with a dataset of Spotify tracks from Kaggle covering over 120 genres and a wide range of audio features [4]. After cleaning, the dataset contains 81,343 unique songs.

A few preprocessing steps are required before building the graph. Rows with missing values are removed, since incomplete audio features cannot be used in similarity calculations. Duplicate songs are also removed. The same track often appears multiple times across releases, so we deduplicate on track name and artist, keeping the most popular version as a tiebreaker. Popularity itself is not used as a feature.

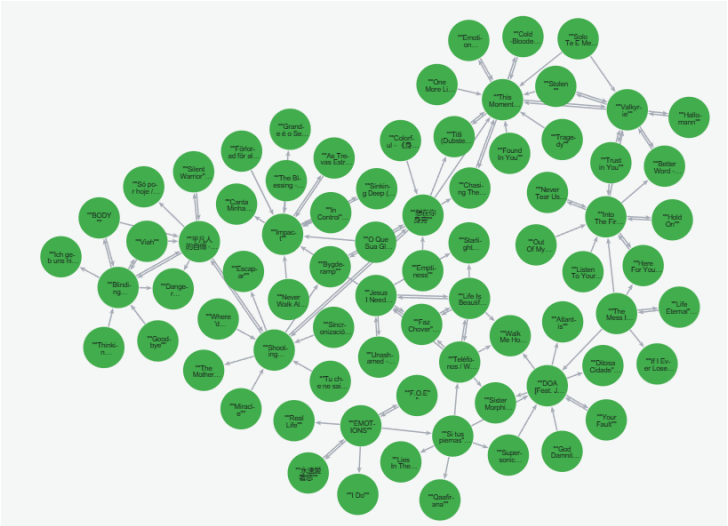
Each song includes 12 audio features, but we use only 7 for similarity: danceability, energy, speechiness, acousticness, instrumentalness, valence, and tempo. These features best reflect how a song sounds to a listener. Features such as key, mode, and time signature are excluded because they do not reliably capture perceptual similarity. Popularity and explicit flags are also excluded to avoid biasing the graph toward well known or non-sonic attributes. All features are standardized before computing cosine similarity so that no single feature dominates due to scale differences.

Solution Overview

The recommendation pipeline runs in three stages, moving from graph construction to graph community and path algorithms, and finally to queue generation:

1. Graph Construction

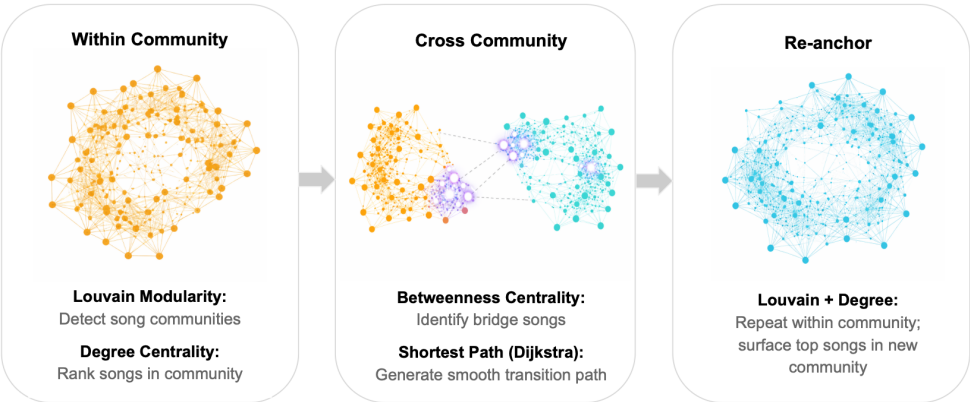
Each song is represented as a node in the graph. Edges are created by connecting each song to its nearest neighbors using cosine similarity across the selected features. We use k-nearest neighbors with k equal to 5, which results in a graph of roughly 80,000 nodes and 400,000 edges. Each edge stores both similarity and distance, where distance is defined as one minus cosine similarity. Distance is later used as the cost for pathfinding.



Graph of songs showing nearest-neighbor connections

2. Graph Community & Path Algorithms

We next apply a sequence of graph algorithms to identify community structure and construct a path from the input song to a new region of the graph:



Louvain Community Detection is used to partition the graph into clusters based on connection density. Each cluster represents a group of songs that are similar in sound and serves as a local region of the graph.

Degree Centrality identifies well connected songs within each community. These tracks act as representative points within the cluster and help anchor the listener in familiar territory at the start of the path.

Betweenness Centrality is computed across the full graph. This measures how often a song lies on shortest paths between other songs. High betweenness songs tend to sit between clusters and act as natural transition points.

We then identify a **target community**. We take an input song, identify its cluster, and compute centroid vectors for all clusters by averaging their feature vectors. The cluster whose centroid is closest to the input cluster defines a target region that is similar but not identical.

To move between clusters, we identify a **bridge song**. A valid bridge must connect to both the input cluster and the target cluster. Among these candidates, we select the song with the highest betweenness centrality, since it is most structurally positioned between the two regions.

Bridge song connecting two communities

3. Queue Generation

With the input song, bridge song, and target community defined, we generate a recommendation as a path through the graph.

We run Dijkstra's algorithm twice using cosine distance as the edge weight. The first run finds the shortest path from the input song to the bridge, moving through the listener's

current community and toward its boundary. The second run finds the shortest path from the bridge to a target song within the target community.

These two paths are concatenated to form a single ordered sequence. Each step represents the smallest available change in sound, resulting in a gradual transition. The final output is a listening queue that moves the user from familiar songs, through a structurally meaningful bridge, and into a new sonic community.

Example
Starting from the track Bohemian Rhapsody, the system first moves through songs within the same rock cluster. As it approaches the boundary, it transitions through a bridge track that shares characteristics with another style. From there, it continues into a new cluster, such as pop or indie, in a way that feels continuous.

The result is an ordered list of songs that gradually shifts genre:

Position in Queue	Track Name	Artist	Community ID
1	Bohemian Rhapsody	Queen	10116
2	O Amigo	Sergio Lopes	10116
3	Miranda	Frank Turner	10116
4	SAYONARA	Aries	10116
5	Burn	Hex Cougar; Pauline Herr	38520
6	big city blues	Lil Peep; Cold Hart	38520
8	Peace To The Mountain	Coheed and Cambria	38520
9	Fa La La	Justin Bieber; Boyz II Men	38520
10	How Long	Tove Lo	38520

In this queue, *Bohemian Rhapsody* is the input song, *Burn* is the bridge song, and *How Long* is the target song.

Discussion
In terms of future improvements: the current implementation selects the target community by centroid proximity and the target song by similarity to the bridge. As play history accumulates, both decisions become candidates for personalization: target community selection could be weighted toward communities the listener has responded

well to historically, and songs already heard could have their edge weights inflated to route the traversal around them.

While this project focuses on the core algorithm, a production version could use Neo4j for graph traversal, MongoDB for song metadata, and Redis for caching and session state.

Conclusion

Music discovery is not just about matching taste - it is about expanding it. By modeling songs as a graph and identifying structural connections between clusters, we can generate recommendations that guide listeners beyond what they already know without breaking the listening experience. This reframes recommendation as navigation through a space, where each step is small, but the overall journey leads somewhere new.

References

[1] Spotify, Spotify Q3 2025 Shareholder Letter (2025), Spotify Investor Relations.
[2] R. Mehrotra, J. McInerney, H. Bouchard, M. Lalmas & F. Diaz, Algorithmic Balancing of Familiarity, Similarity & Discovery in Music Recommendations (2020), Spotify Research.
[3] A. Anderson, L. Maystre, I. Anderson, R. Mehrotra & M. Lalmas, Algorithmic Effects on the Diversity of Consumption on Spotify (2020), Proceedings of The Web Conference 2020.
[4] M. Pandya, Spotify Tracks Dataset (2022), Kaggle.